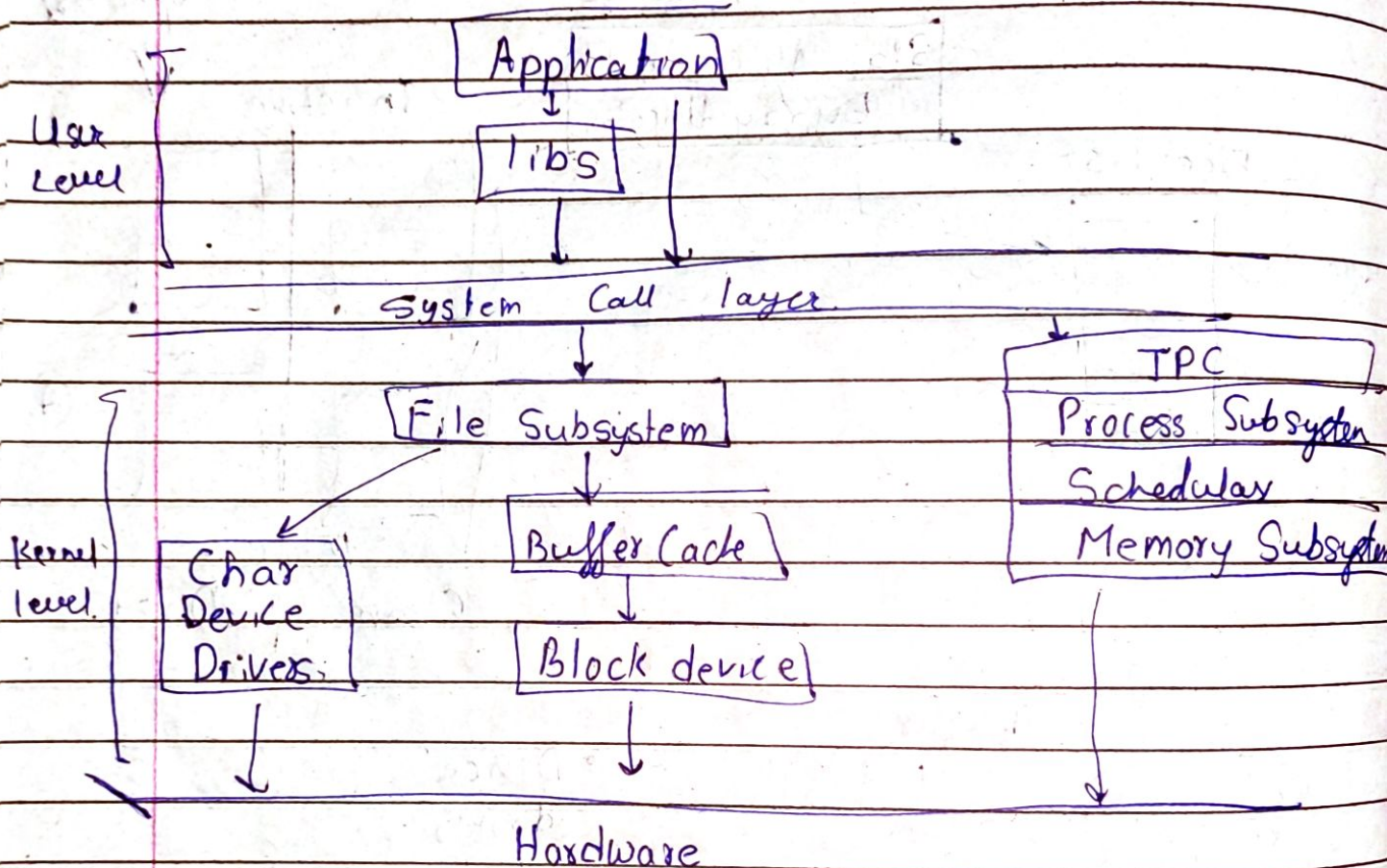


GTK Library

* Kernel Architecture

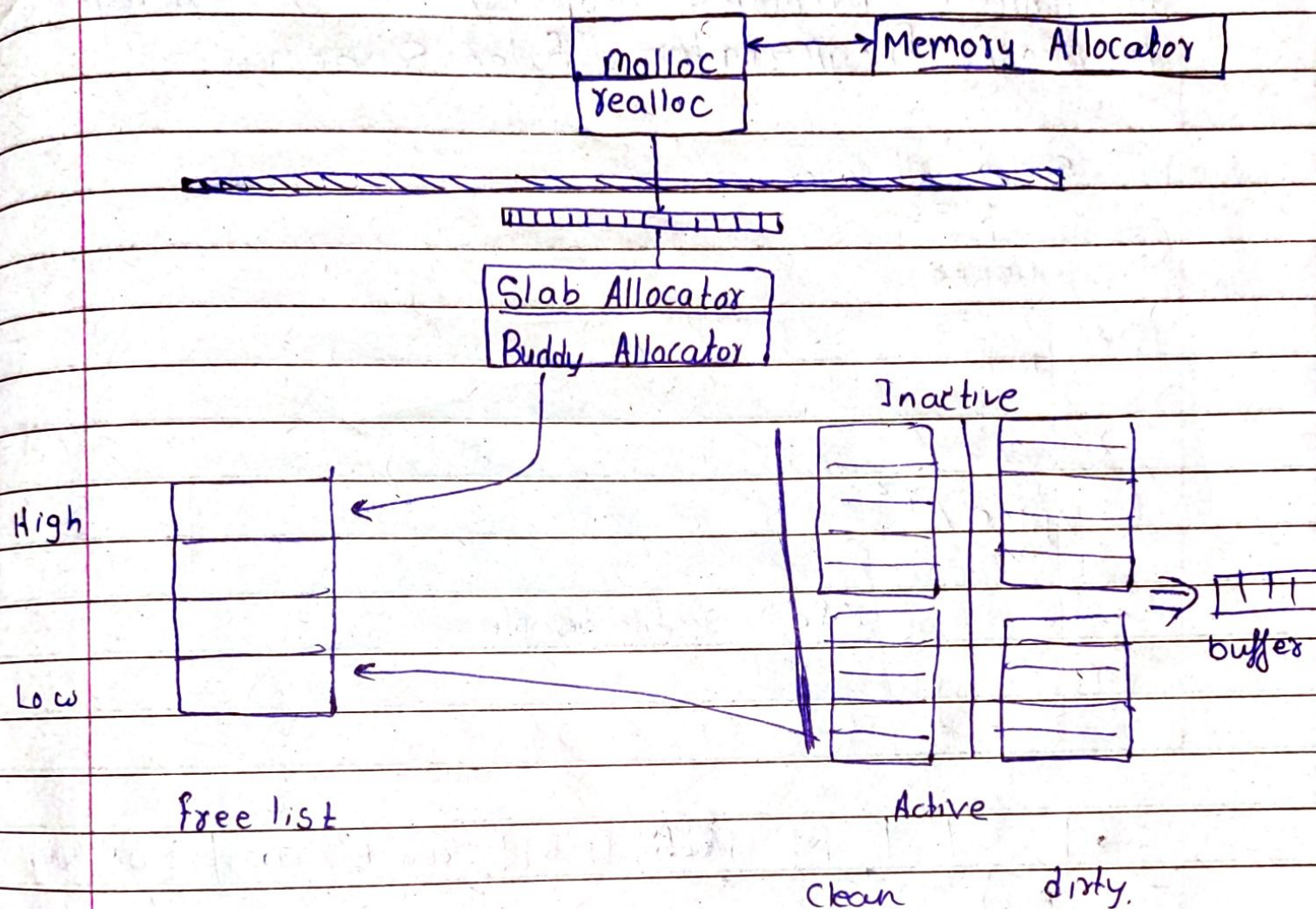


Database Engine

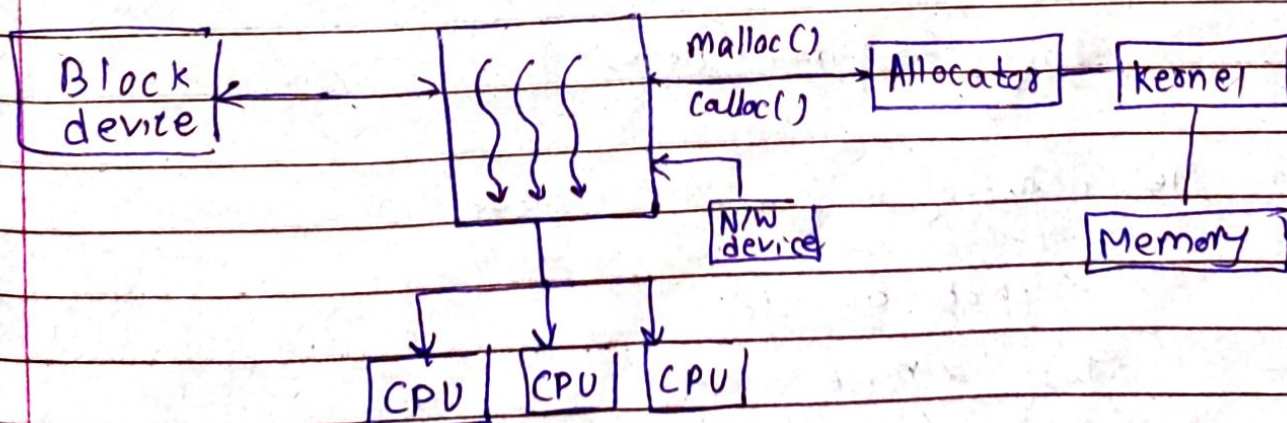
Q1 How to prevent deadlock?

ans Mutual exclusion

* Memory Subsystem



* Functional block diagram of process.



Why programs fail.

multithreaded programs, heap contention, heap leakage,
memory fragmentation, false sharing

System Call layer
VFS

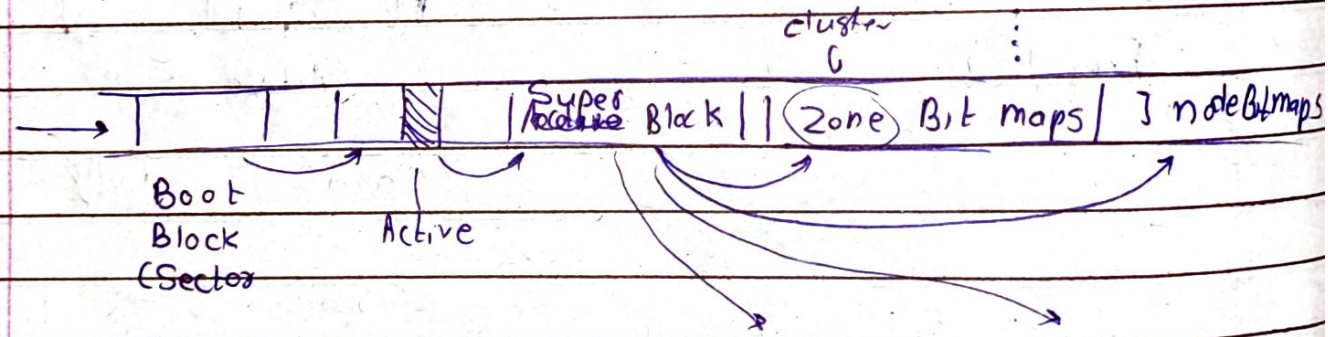
USBFS

TMPFS

NTFS

procfs
/proc / 1 ← Pid

/fd ← file descriptor



Motherboard. goes.

Int 13h Hardware interrupt

→ Inode List / first Data Block

Inode → index node.

Metadata of a file → Pnode

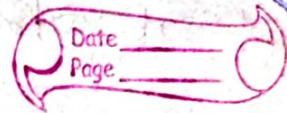
/ root direction

/bin	/mnt	/usr
/boot	/lib	/tmp
/dev	/opt	/var
/etc	/sbin	/

name of kernel image file.

→ ez image,

where is kernel image residing



What is use of bitmap

Superblock

It has pointed to many other entities.
(Minix) → first OS used by Linux

data block size
→ 1Kb

→ Total no of data blocks

" " " Zones

" " " Inode Bitmap blocks.

" " " Zone bitmap blocks.

" " " Free Inodes

" " " Zones (Data Block)

First Data Zone

Maximum file size

Magic No Ext2 0xEF53

Pointer to Inode Bitmap

" " " "

" " " "

Pointer to Zone Bitmap

" " " "

" " " "

Inode of mounted file system

Inode of " " " " on

Time of last update

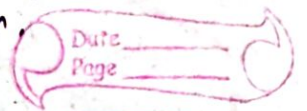
File System mounted as read only.

Dirty flag.

mkefs - make file system

fsck → File System Scan.

Inode is metadata related to file
it stores additional information.



Magic no is set of 2 bytes.

14 15 05 03

What is EF53, convert to bits

EF53

1111
8427

1110 1111 0101 0011

* Inode:

Mode - Type of file, Permissions.

UID, GID

File Size.

Time of last modification mtime

" " " Access atime

" " " Inode changed ctime

Links

Pointer to first Data block

' , 2nd ,
' ,
' ,
' , 10th ,

Single Indirect Pointer

Double Indirect Pointer.

Triple Indirect Pointer

Status of Incore Inode

- Inode locked
- A process waiting for Inode.
- Incore Inode different from disk.
- " File
- File is mount point
- Logical device no of File Systems

Inode no
pointers to other Inodes
Reference Count

ls -l <file.name> → get inode no of file.

Q What is inode of root -
whose inode no is 1.

Date _____
Page _____

Regular file, directory, Symbolic link file, device file, binary file.

In -s /usr /Sec /kernel → new file s.t.t.
(create shortcut)

/dev /tty0 → device file (binary file)

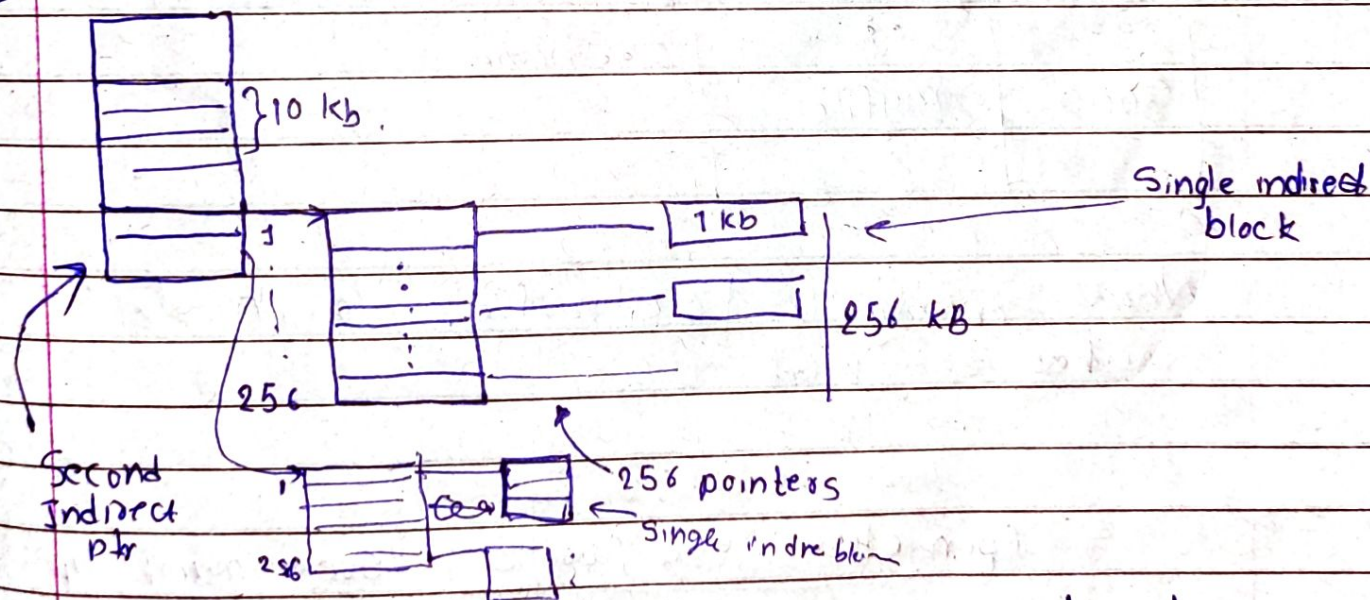
↑
It starts for first terminal or console.

For dev binary, Sbm

Q Where are executable files stored

loop back device

inode structure



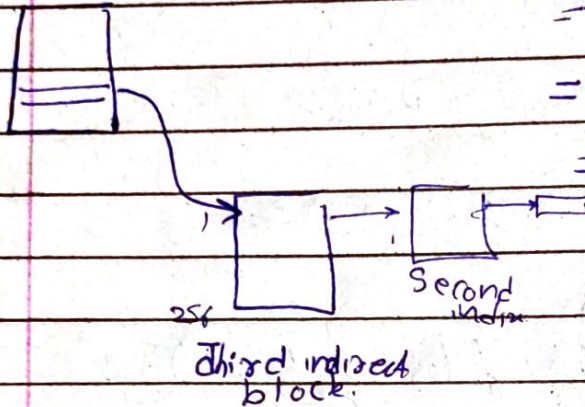
- Each pointer points 1kb data so, single indirect block has 256 kb data.
- If any file size is greater than 256 kb then second indirect pointer points to second indirect block.
- each pointer in second indirect block points to single indirect block.

$$2^8 \times 2^8 \times 2^8 \times 1 \text{ kb}$$

$$2^{24} \times 1 \text{ kb}$$

$$64 \times 2^{20} \times 1 \text{ kb}$$

So second Indirect block refers
64 mb data.



$$= 2^8 \times 2^8 \times 256 \text{ kb} \leftarrow \text{mb}$$

$$= 256 \text{ kb} \times 64 \text{ mb}$$

$$= 16 \text{ GB.}$$

/home/mydir/myfile

Inode	Name
200	...
100	...
3000	myfile
	...

mydir.
← . Current directory
.. home

When ~~rm~~ myfile 00 | myfile file, inode
number is made 00

Open close read write lseek mount dup
name

iget lalloc iput ifree
alloc free

getblk brelsc bread breado.
bmap bwrite

access time of secondary device $\rightarrow$ in terms of milliseconds,

Date
Page

Q from where kernel can get inode number of file from file name.
directory, inode list table, datablock of file

Algorithm iget

I/P - file name $\rightarrow$ inode

O/P - locked inode

{

while (not done)

 & if (inode in Inode Cache)

 & if (inode locked) &

 Sleep (wakeup when unlocked)

 }

 if (inode in unused list) remove from list.

 ++reference count

 Set lock bit

 return inode.

 }

read inode (Algo - broad)

Initialize mod - ++reference count

put inode in Inode table.

Set lock bit.

return inode;

}

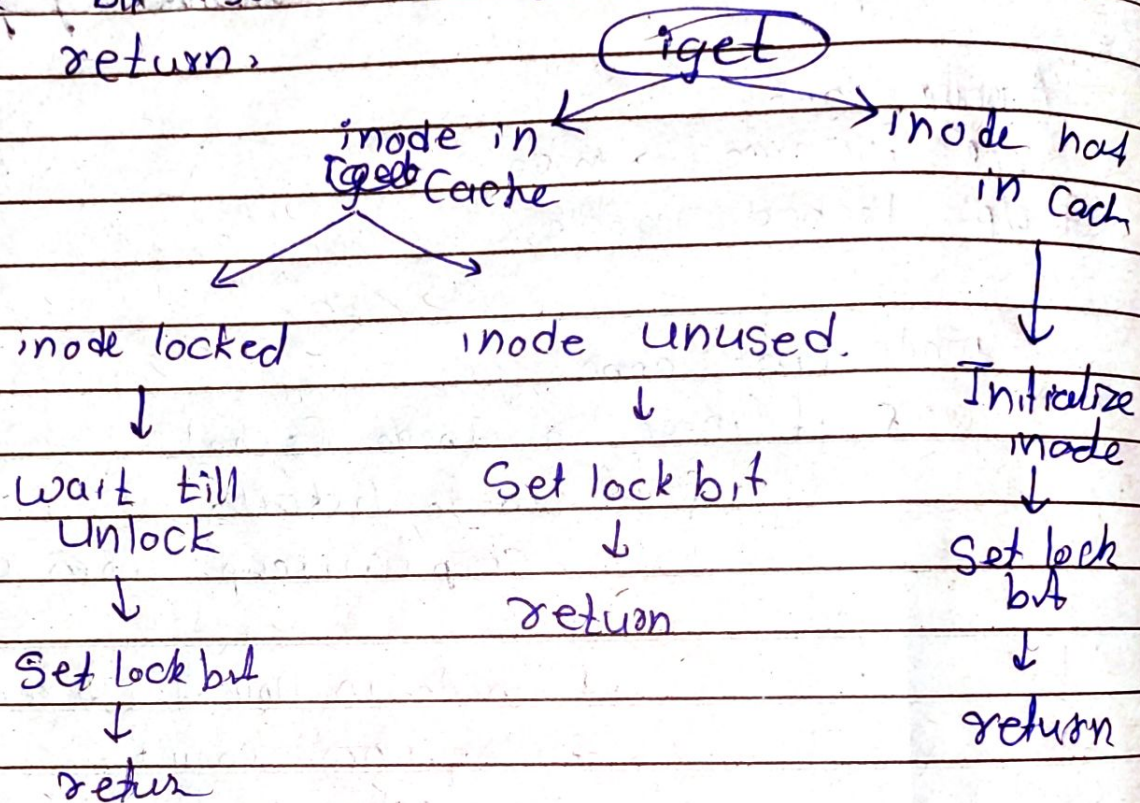
}

Busy Waiting Approach

Can be used to replace Mutex locks

If inode is in inode cache $\Rightarrow$ The file was used earlier.

- If someone is using inode it is locked. So wait till it gets released or unlocked.
- If no one is using inode it will be in unused list. Then remove the inode from unused list and set lock bit. So no one can use it. and return.



Algorithm ~~iget~~ input

J/P → pointer to Incore inode.

O/P → none.

{

lock the inode if not locked

-- reference count

if (reference count == 0)

{

if (linkcount == 0) {

Free Datablocks - Algo - free

Set file type to zero

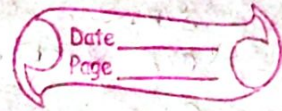
free inode - Algo - free

return

}

if (file accessed / modified / inode changed)

Update Inode on disk.



put inode on Unused list.

release inode lock.

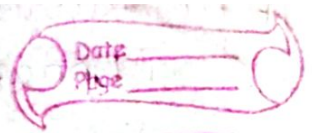
- ifut is called while closing a file
- If ref count is not zero we just release the inode lock and the inode remains in inode table as there is chance of it getting referenced again.
- If link count is 0 you want to delete the file
- If link count is not zero -
that means no process is currently using inode we put it in unused list.
- If the inode was change or file was accessed or modified we update the temporary inode on disk with its temporary copy from memory RAM So that the changes are made permanent.
- If link count is 0 we call ifree to remove inode from inode table permanently.

Why is file type 0 used → setting inode number in directory entry

Algorithm - ialloc.

I/P -

O/P - locked Inode.



```
{ while (not done)
```

```
{
```

```
  if (Super block locked)
```

```
  {
```

```
    Sleep (wakeup when Superblock unlocked)
```

```
  }
```

```
  if (free Inode List empty)
```

```
  {
```

```
    lock the Super block.
```

```
    • get the last remembered Inode for free  
      Inodes Search
```

```
    • Search for free Inodes from disk till  
      free Inodes list is not full or no  
      more free Inodes.
```

```
    • if (nofree - Inodes) return noFree Inodes
```

```
    • set last remembered Inode field.
```

```
    • Unlock Superblock.
```

```
  }
```

```
  Get inode from free Inode List.
```

```
  Initialize Inode - Zero Inode Buffer Contents.
```

```
    - ++ Reference Count.
```

```
    - Put Inode in Inode Table
```

```
    - Set lock bit
```

```
  Write Inode to Disk
```

```
  { -- No of Free Inodes in Superblocks  
    Make entry in free Inode Bitmap
```

```
    Unlock Superblock
```

```
  return InodeLocked
```

```
}
```

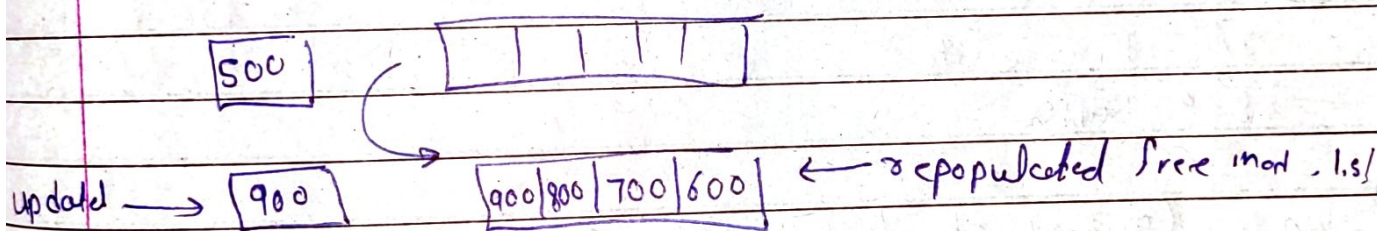



500	400	300	200	100
-----	-----	-----	-----	-----

Free Inode List

Inode Table

- Instead of finding free inode from disk we will place some inodes in free inode list.
- This list is smaller than that of disk.
- So when we need an inode we can get from free inode list instead of disk.
- When list is empty we find free inodes from disk after last remembered inode.
- When we populate list again we update the last remembered inode.



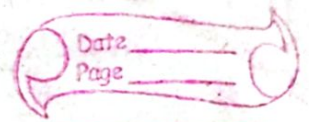
- Free inode list is present in memory

Where will we search free inode on the disk.

Algorithm - ifree.

I/P - Inode J

O/P - None.



```
{
  if (Superblock locked) { Sleep (wakeup on Superblock Unlocked) }
  if (Free Inode List full) {
    if (Inode J < Last Remembered Inode) {
      Last Remembered Inode ← J.
    }
  }
  else {
    Store Inode J on Free List.
    Make entry in Free Inode Bitmap
    ++ Free Inodes in Superblock.
    return.
  }
}
```

Algo - alloc

I/P - File System

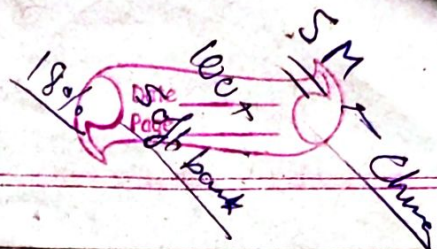
O/P - Buffer.

```
{
  while (Superblock locked) { Sleep (wakeup upon Unlocked) }
  remove the block from the List of Free disk blocks.
  if (removed Last block from the list) {
    lock Superblock
    read block (Algo - bread)
    copy from buffer to List
    release buffer - Algo - brelease.
    Unlock Superblock.
  }
  get buffer for block - Algo getBlk.
  zero buffer contents
  -- free Disk Blocks
  Mark Superblock as modified
  return buffer.
}
```


It is used to allocate disk block

400 300 200 100

Free disk block



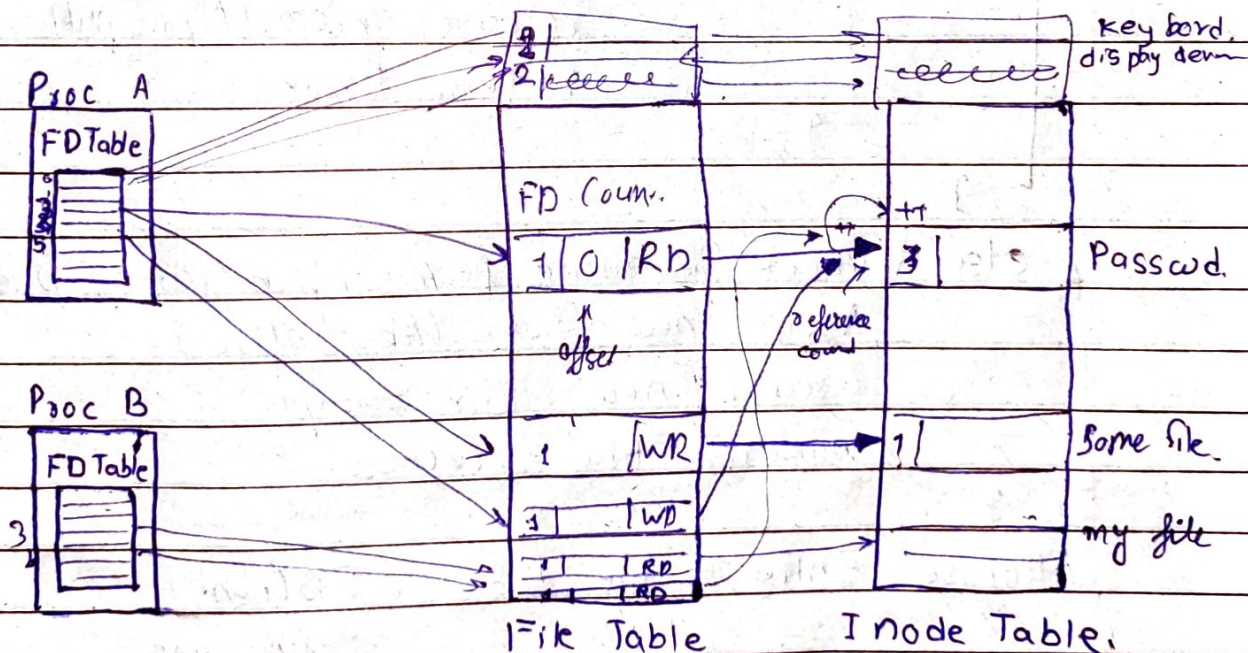
* open("/etc/pass

Proc A

```
int fd1 = open("/etc/password", O_RDONLY);
int fd2 = open("Somefile", O_WRONLY);
int fd3 = open("etc/password", O_WRONLY);
```

Proc B

```
int fd1 = open("myfile", O_RDONLY);
int fd2 = open("/etc/password", O_RDONLY);
```



FDTable contains pointer to current processes
 For example in Proc A when line one is executed
 when we open a file an entry is made in file
 table and reference
 count is increased in node 500,000,000,000,000
 table

entries

0, 1, 2 are not used in FD table why?

0 → stdin

1 → stdout

2 → stderr.

create(char *Filename, int mode);

I/P - Filename, permissions

O/P - File Descriptor.

{

get_inode_no(Algo_name);

if (file already exists) {

if (no write permission in Parent Dir) {

release_inode(Algo_iput)

return Error

}

}

else { Check for write permission in parent Directory

Assign new inode (Algo_ialloc)

Create new Dir Entry.

wakeup the processes

}

Allocate Entry in file table - FDCount

Permissions

file offset

Pointer to inode.

if (file existed at creation time) {

free disk blocks (Algo_free)

}

Set owner group

File Size $\leftarrow 0$

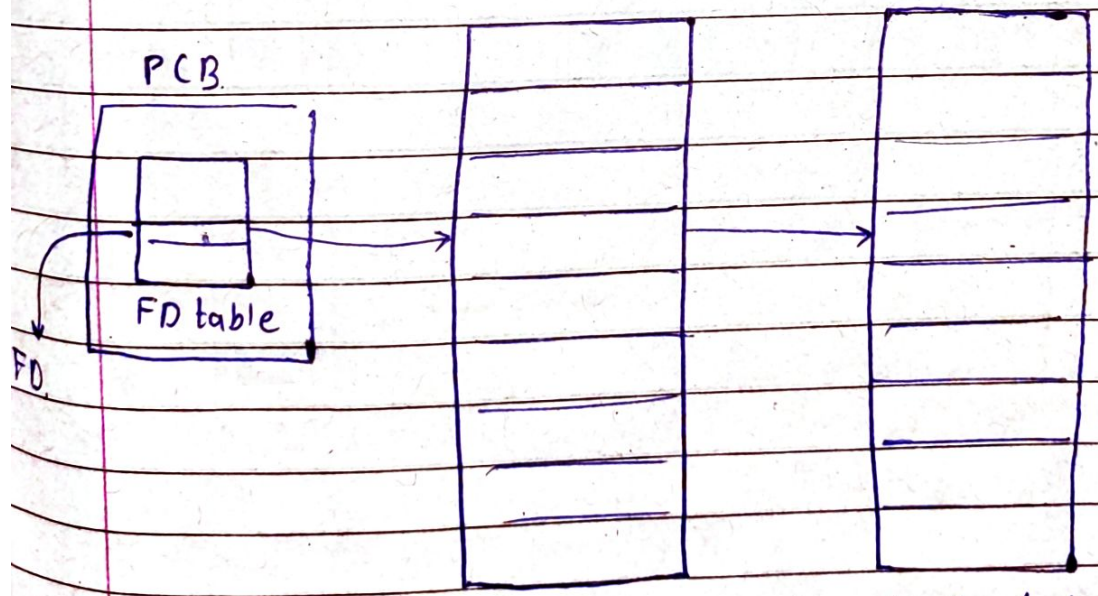
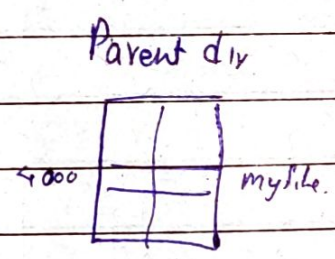
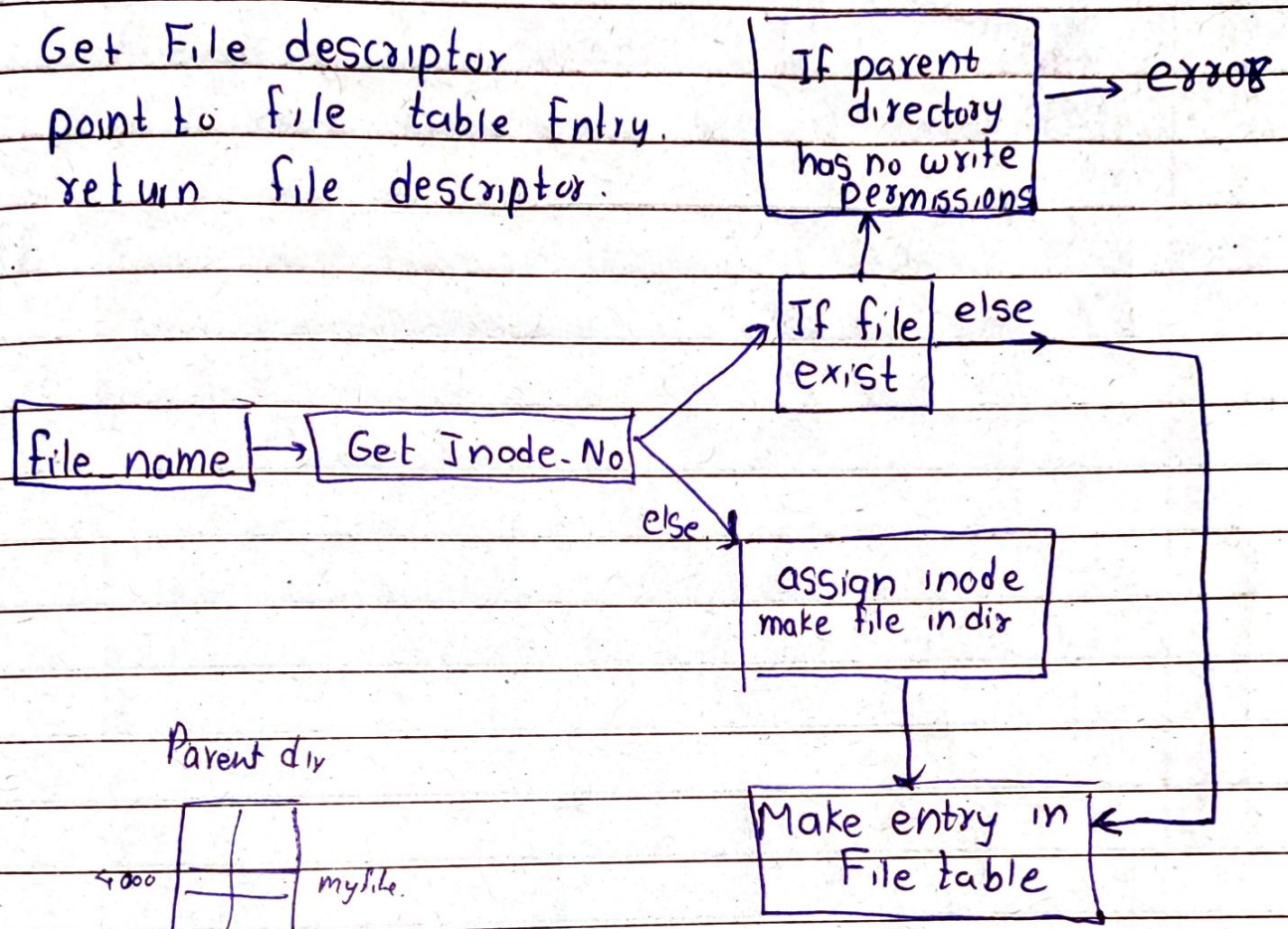
Unlock Inode

Get File descriptor

point to file table Entry.

return file descriptor.

}



File Table

inode table

first we create an entry in file table then we make entry in inode table then we return the file description from FD table points to file table

Name of inode table in Linux kernel

" File Table

" PCM

Date _____
Page _____